

Topic 10.3

10.3: Run Time Type Identification (RTTI)

Some compilers provide the capability to determine the type of object a pointer points to at runtime. This is known as Run Time Type Identification.

Here is an example code fragment:

```
Mammal *m; // m can point at Mammal
           // or anything derived.

    if (typeid(*m) == typeid(Mammal))
    ...
    else if (typeid(*m) == typeid(Dog))
    ...
```

In general, RTTI is undesirable and indicates poor design and inadequate use of virtual functions. For example, take the code from the `if` statement that is executed when `m` points to a `Mammal` and put it in a virtual function in the base class `Mammal`. Then take the code from the `if` statement that is executed when `m` points to a `Dog` and put it in a virtual function of the same name in the `Dog` class.

RTTI is not supported by all compilers.

The above code fragment indicates a common abuse of RTTI: using it as a "switch" statement instead of using virtual functions appropriately. If additional derivations are made, the above code fragment may cease to work correctly. If additional derived classes are introduced, the above code fragment may cease to work correctly because it then would not account for all possible derived classes in the hierarchy.

RTTI may decrease the efficiency of the executable code significantly. The reason is, the compiler has to generate "symbol table" information that is managed at runtime. Unfortunately, the compiler cannot always know when RTTI will be used and when it won't, so it has to keep the information around all the time, whether it is used or not.

If you really want to use RTTI, it may be best to build your *own* for portability reasons. The following fragment indicates how to do this.

```
enum MammalType {mammal, dog};

class Mammal
{
public:
    // Constructor sets the type.

    Mammal() : my_type(mammal) {};

    // Accessor function retrieves the type.

    virtual MammalType GetType() { return
```

```
        my_type; };

    private:
        MammalType my_type;
}

class Dog : public Mammal
{
    public:

        // Constructor sets the type.

        Dog() : my_type(dog) {};

        // Accessor function retrieves the type.

        virtual MammalType GetType() {return my_type; };

    private:
        MammalType my_type;
}

Mammal *m;

if (m->GetType() == mammal)
    ...
else if (m->GetType() == dog)
    ...
```

In doing this, you are doing essentially what the compiler would do if you relied on compiler-supported RTTI. However, you only incur the expense where you really need it.

Topic 10.4 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-10-dot-4>).